

Semantic Drift, Not Rank Collapse: Why Stable Coordinates Matter for Learning in Joint-Embedding Spaces

Andrey Lazarev Independent Researcher | April 2026

Abstract

Representational collapse in Joint-Embedding Predictive Architectures (JEPAs) is conventionally attributed to rank degeneration of the latent space and addressed through geometric regularization (VICReg, SIGReg). We show that this explanation is incomplete. In a controlled Push-T world-model setting with gym-pusht physics, we demonstrate that a free encoder regularized by SIGReg maintains full rank (effective rank 2.99/3) and high isotropy (0.86) — yet produces predictions more than an order of magnitude worse than a prescribed encoder with lower rank and isotropy.

We identify the actual failure mode: temporal non-identifiability of the representation. In the first epochs of training, the free encoder restructures its coordinate system so rapidly that a linear decoder trained at epoch t produces catastrophically wrong outputs at epoch $t+1$ ($R^2 = -62$, averaged across seeds). All target variables are normalized to unit variance; negative R^2 therefore indicates performance worse than predicting the mean. Procrustes alignment confirms that 80% of this change is structural, not rotational. A causal freeze test shows that stabilizing the encoder at epoch 1 improves prediction by 20%. However, the stabilized free encoder remains more than $25\times$ worse than prescribed — demonstrating that coordinate stability alone is insufficient without semantic alignment with the task.

A random fixed encoder — a frozen random orthogonal projection with no semantic content — outperforms the free encoder by $17\times$, confirming that coordinate stability alone provides a substantial advantage even without task-relevant alignment. A completed 2×2 factorial design reveals that the two factors — stability and alignment — are not independent: an encoder initialized on the exact ground-truth coordinates but allowed to drift performs no better than free (and sometimes worse). Stability is a prerequisite; alignment contributes only when stability is maintained.

These findings establish, through direct experimental controls, that the advantage of prescribed representations involves at least two contributing factors: temporal stability of coordinates and their alignment with task-relevant quantities. In this setting, the collapse problem is not about preserving geometric properties of the space, but about maintaining an identifiable and stable coordinate system in which downstream modules can accumulate learning. Additional experiments confirm that drift is not a tuning artifact (prescribed wins at all four tested learning rates, $4.3\text{--}7.0\times$), that EMA target encoders do not fix the problem ($6.1\times$ worse than prescribed), and that the same drift pattern appears in vision (Rico UI) — confirming cross-modal generality.

Three seeds, 30 epochs, 200 episodes of real physics data per seed. Code and data: <https://github.com/revenue7-eng/prescribed-axes>

1. Introduction

JEPA architectures learn by predicting masked representations from visible context, without pixel reconstruction (LeCun, 2022; Assran et al., 2023). The central vulnerability: a free encoder can minimize the prediction loss by mapping all inputs to a constant, collapsing the representation.

Two families of solutions dominate the current discussion. Regularization-based approaches (VICReg, Bardes et al., 2022; SIGReg, Balestrieri & LeCun, 2025) penalize geometric degeneration — low variance, high covariance, non-Gaussian marginals. Generative approaches (PAN, Ozkara et al., 2025) supplement prediction with pixel reconstruction. Both assume the problem is how the model trains: the loss function either fails to prevent collapse or fails to ground the representations.

Recent work on prescribed axes (Lazarev, 2026) proposed a different framing: the problem is where the model trains. If the axes of the latent space are fixed before training — from physical coordinates, semantic features, or cluster centroids — collapse is structurally impossible, and regularization is unnecessary. Four experiments across modalities showed 5–38 $\times$ improvements over free baselines.

However, this framing raises a question: if prescribed axes prevent collapse, what exactly goes wrong in free space? How does the representation evolve during training — and whether this evolution preserves a consistent coordinate system for downstream modules? We assumed rank collapse. The data shows otherwise.

Contributions:

1. We show that geometric metrics (rank, isotropy) do not explain the performance gap between prescribed and free representations: a free encoder with rank 2.99 and isotropy 0.86 loses to a prescribed encoder with rank 2.91 and isotropy 0.66 by more than 30 $\times$ under identical training conditions.
2. We identify a phase of temporal non-identifiability in free encoders: during epochs 0–2, the coordinate system changes so rapidly that linear decoders become invalid within one training step (R^2 transfer < -16 across all seeds).
3. We provide direct interventional evidence that this instability degrades learning: freezing the encoder at epoch 1 improves prediction by 20% compared to continued joint training.
4. We decompose the advantage of prescribed representations into two factors — coordinate stability and semantic alignment — showing that neither alone explains the full effect.
5. We isolate these factors with a completed 2×2 factorial design. A random fixed encoder (no semantic content) outperforms free by 17 $\times$. An aligned-but-drifting encoder (initialized on ground-truth coordinates, then allowed to train) performs no better than free. This establishes that stability is a prerequisite for alignment to have any effect, not an independent contributing factor.

2. Background and Setup

2.1 Problem Setting

We study a minimal LeWM world model (Maes et al., 2026) on the Push-T environment: a planar pushing task where a robot arm pushes a T-shaped block. The state is 5-dimensional: agent position (x_a, y_a) and block pose (x_b, y_b, θ_b). The world model predicts the next latent state from a context window of $H=3$ previous states and actions.

Prescribed encoder $h(s) = \text{normalize}(s[2:5])$: extracts and normalizes the block coordinates (x_b, y_b, θ_b). No learnable parameters.

Free encoder $f_\theta(s)$: an MLP ($5 \rightarrow 64 \rightarrow 64 \rightarrow 3$) that learns a 3D embedding end-to-end with the predictor. Regularized by SIGReg (Balestriero & LeCun, 2025) to prevent rank collapse.

Both encoders produce 3D representations. The predictor architecture, loss function, optimizer, and training schedule are identical. The only difference is the encoder.

2.2 Experimental Protocol

Three seeds (42, 123, 777). 200 episodes collected via gym-pusht with real pymunk physics. 30 training epochs. SIGReg regularization coefficient $\lambda=0.09$. All results averaged across seeds with individual seed values reported.

2.3 Metrics

Covariance analysis. At sampled epochs, we compute the covariance matrix of all validation embeddings, its eigenspectrum, effective rank (exponential of the normalized eigenvalue entropy), condition number, and isotropy (ratio of smallest to largest eigenvalue).

Drift metrics. Between consecutive epochs t and $t+1$, we compute: - **Raw drift**: mean ℓ_2 distance of the same input’s embedding across epochs - **Aligned drift**: residual after optimal Procrustes alignment (rotation removed) - **R^2 transfer**: a linear probe h_t trained to map embeddings at epoch t to ground-truth (x, y, θ) is applied to embeddings at epoch $t+1$. $R^2 < 0$ indicates the mapping has become worse than a constant predictor.

Freeze test. The free encoder is trained jointly with the predictor until epoch T , then frozen. The predictor continues training alone for the remaining epochs. Comparison with unfrozen baseline and prescribed baseline isolates the causal effect of encoder stability.

3. Why Geometry Is Not Enough

The standard explanation for free encoder underperformance is rank collapse: the encoder maps inputs to a low-dimensional subspace, starving the predictor of information. SIGReg prevents this by penalizing deviations from an isotropic Gaussian.

Our data contradicts this explanation.

Observation 1: Full rank, poor performance. The free encoder with SIGReg achieves effective rank 2.99/3 and isotropy 0.83–0.90 across seeds. The prescribed encoder has lower rank (2.91–2.99) and lower isotropy (0.53–0.79). Yet prescribed outperforms free by more than an order of magnitude in prediction loss (0.0025 vs 0.082, averaged across seeds).

Observation 2: Anisotropy reflects task structure. The prescribed eigenvalue spectrum [0.098, 0.078, 0.064] is anisotropic because the physical quantities (x , y , θ) have different ranges and dynamics. SIGReg forces the free encoder toward isotropy [0.98, 0.93, 0.85] — a uniform geometry that does not match the task. The prescribed anisotropy is not a defect; it reflects the structure of what is being predicted.

These observations rule out rank collapse as the explanation. The free encoder maintains full rank and high isotropy — the geometric properties that regularization is designed to ensure — and still loses by an order of magnitude.

4. Temporal Drift in Representation Space

If geometry does not explain the gap, what does? We track how the free encoder’s coordinate system changes during training.

4.1 Large-Scale Structural Drift in Early Training

In the first epoch of training, the free encoder restructures its entire coordinate system. Raw drift on epoch 0→1 ranges from 1.28 to 1.57 across seeds (measured as mean ℓ_2 displacement of validation embeddings). By comparison, prescribed drift is exactly 0.000000 on every epoch — the encoder is fixed.

After Procrustes alignment (removing optimal rotation, reflection, and uniform scaling), 79% of the epoch 0→1 drift remains. Procrustes does not remove nonlinear deformations or local geometric distortions, so this 79% is an upper bound on structural change — but even as an upper bound, it indicates that most of the coordinate reorganization is not explained by a global rotation.

By epoch 3, raw drift drops below 0.14, and the residual after Procrustes drops further — late-epoch changes are increasingly rotational rather than structural. The large-scale drift phase is confined to epochs 0–2.

4.2 Decoder Invalidation

The most direct measure of temporal non-identifiability: a linear probe trained to decode ground-truth (x , y , θ) from embeddings at epoch t is applied to embeddings at epoch $t+1$.

R^2 transfer values on epoch 0→1: - Seed 42: −16.9 - Seed 123: −62.2 - Seed 777: −25.4

All deeply negative. A linear decoder trained on the encoder’s current coordinate system produces outputs worse than a constant predictor after a single epoch of encoder training. The coordinate system has changed so completely that any fixed mapping from it to the physical world is invalidated.

By epoch 2→3, R^2 transfer recovers to 0.73–0.76 across seeds — the coordinate system has stabilized. But the predictor has already spent its critical early training phase building on a representation space that was moving.

4.3 Implication

The predictor and the encoder train simultaneously. The predictor builds its weights assuming the encoder’s coordinate system. When that coordinate system changes faster than the predictor can adapt — which our data shows happens in epochs 0–2 — the predictor’s gradients become incoherent. It is learning to predict in a space that will not exist on the next step.

Prescribed axes eliminate this phase entirely. The coordinate system is stable from epoch 0. Every gradient step the predictor takes is cumulative.

5. Does Drift Matter? Interventional Evidence

Observation alone does not establish causation. The drift might be a symptom of learning, not a cause of poor performance. To test this, we freeze the encoder at various epochs and continue training only the predictor.

5.1 Freeze Test Results

Condition	Seed 42	Seed 123	Seed 777	Mean	vs unfrozen
Prescribed	0.0022	0.0030	0.0023	0.0025	—
Free unfrozen	0.0864	0.0832	0.0738	0.0811	baseline
Free freeze@1	0.0794	0.0501	0.0651	0.0649	+20.1%
Free freeze@2	0.0869	0.0776	0.0718	0.0788	+2.9%
Free freeze@3	0.0818	0.0773	0.0690	0.0760	+6.3%
Free freeze@5	0.0796	0.0774	0.0679	0.0750	+7.6%
Free freeze@7	0.0824	0.0833	0.0681	0.0779	+3.9%
Free freeze@10	0.0822	0.0869	0.0770	0.0820	−1.1%

5.2 Interpretation

Freezing the encoder at epoch 1 — before the large-scale drift phase — improves prediction by 20% compared to letting the encoder continue training. This provides direct interventional evidence that stabilizing the representation improves learning.

Freezing at epoch 2 gives only +2.9%, and freezing at epoch 10 gives −1.1% (neutral). This pattern makes sense: by epoch 2, the encoder has already undergone its large-

scale restructuring, and the damage is done. By epoch 10, the encoder has stabilized and freezing neither helps nor hurts.

5.3 Stability Is Not Sufficient

The freeze@1 result (0.065) is 20% better than unfrozen (0.081) — but still more than 25× worse than prescribed (0.0025). Stabilizing the coordinate system helps, but the stabilized coordinates are not aligned with the task.

This decomposition is the central result. The prediction error can be understood as a function of two independent factors:

$$\mathbf{E} \approx \mathbf{g}(\text{stability}) \cdot \mathbf{h}(\text{alignment})$$

where $\mathbf{g}$ captures the cost of coordinate drift and $\mathbf{h}$ captures the cost of misalignment with task-relevant quantities. The freeze test provides initial evidence for this decomposition (freeze@1 reduces error by 20%), but does not cleanly separate stability from learned content. Section 5.4 provides the definitive test using a random fixed encoder that isolates pure stability. Two factors are required:

1. **Temporal stability:** the coordinate system must not change faster than the predictor can adapt. Effect: 17× (measured by random fixed encoder control, Section 5.4).
2. **Semantic alignment:** the coordinates must correspond to task-relevant quantities. Effect: 13× (measured by the gap between random fixed and prescribed).

Neither factor alone explains the full gap (>30× under identical training conditions). Together, they do.

5.4 Isolating Stability from Alignment: Random Fixed Encoder

The freeze test demonstrates that stability helps, but it confounds two variables: the frozen free encoder is stable *and* has already learned a partial representation during epoch 1. A stronger test isolates stability by removing any learned content.

Random fixed encoder. We construct an encoder with no semantic content: a random orthogonal projection from the normalized 5D state to 3D, frozen from initialization. This encoder has perfect temporal stability (the coordinate system never changes) but zero alignment with task-relevant quantities (the axes are random linear combinations of all state variables, including agent position). A caveat: because this encoder projects from a different input subspace (all 5 dimensions) than prescribed (only block coordinates), its advantage over free may partially reflect favorable conditioning of the random projection rather than pure stability. The rotated prescribed encoder (below) partially controls for this by using the same 3D subspace.

Rotated prescribed encoder. As an additional control, we rotate the prescribed coordinates by a random orthogonal matrix. This encoder is stable and aligned with the task-relevant subspace, but its axes are not interpretable.

Condition	Seed 42	Seed 123	Seed 777	Mean	vs free
Prescribed	0.000048	0.000043	0.000016	0.000036	233×
Rotated	0.000050	0.000043	0.000023	0.000039	212×
prescribed					
Random fixed	0.000522	0.000446	0.000460	0.000476	17.4×
Free (drifting)	0.008136	0.008988	0.007722	0.008282	baseline

Result 1: Stability alone provides a 17× advantage. The random fixed encoder, with no task-relevant information in its axes, outperforms the free encoder by 17.4× across all three seeds. This is not a marginal effect — it is more than an order of magnitude. A random but stable coordinate system is vastly better than a learned but drifting one.

Result 2: Alignment provides an additional 13× advantage. The gap between random fixed (0.000476) and prescribed (0.000036) is 13.4×. This is the cost of projecting onto the wrong subspace, even when the coordinate system is perfectly stable.

Result 3: Interpretability is irrelevant. Rotated prescribed (0.000039) performs identically to prescribed (0.000036), ratio 1.09×. The predictor does not need interpretable axes — it needs stable ones in the right subspace.

Approximate multiplicative decomposition. The total gap between free and prescribed is 233×. In this configuration, it decomposes approximately into stability (17.4×) × alignment (13.4×) = 233×. However, this factorization relies on three data points, not a full factorial design. Section 5.5 fills the missing cell.

5.5 The Missing Cell: Aligned but Drifting

The 2×2 factorial design requires one more condition: an encoder that starts aligned with the task but is allowed to drift. We test two variants:

Aligned-drifting (linear). A single linear layer (5→3), initialized exactly as the prescribed mapping: weight matrix extracts (x_b, y_b, θ_b) and applies the same normalization. Then trained jointly with the predictor — the encoder is free to drift away from its initialization.

Aligned-drifting (MLP). The same architecture as the free encoder (5→64→64→3), but pre-trained for 200 steps to reproduce prescribed outputs before joint training begins.

Condition	Seed 42	Seed 123	Seed 777	Mean	vs free
Prescribed	0.000049	0.000043	0.000026	0.000039	212×
(stable + aligned)					
Random fixed	0.000522	0.000446	0.000452	0.000473	17.5×
(stable, no alignment)					

Condition	Seed 42	Seed 123	Seed 777	Mean	vs free
Aligned-drifting linear	0.011111	0.012090	0.015347	0.012849	0.64× (worse)
Aligned-drifting MLP	0.008392	0.008429	0.008319	0.008380	0.99× (≈ free)
Free (drifting, no alignment)	0.008136	0.008988	0.007722	0.008282	baseline

Result: Alignment without stability is worthless. Both aligned-but-drifting encoders perform no better than free — and the linear variant is actually 1.55× worse. An encoder initialized on the exact ground-truth coordinates, with the exact correct normalization, loses its entire advantage once it is allowed to train. The perfect initialization is erased by drift.

The factors are not independent. The completed factorial design:

	Stable	Unstable
Aligned	0.000039 (prescribed)	0.012849 (aligned-drifting)
Not aligned	0.000473 (random fixed)	0.008282 (free)

Stability effect among aligned encoders: 330×. Among unaligned: 17.5×. If the factors were independent, these numbers would be similar. They differ by 19×. The interaction is massive: stability matters far more when alignment is present, because alignment is fragile — it exists only as long as the coordinate system that carries it remains fixed. Destroy the coordinate system, and alignment is destroyed with it.

Revised interpretation. The original decomposition (stability 17× × alignment 13× = 233×) was a useful approximation, but the factorial design reveals a different structure. Stability is not one of two independent factors — it is a prerequisite. Alignment contributes an additional 12× *on top of stability* (random fixed → prescribed), but contributes nothing without it (free ≈ aligned-drifting). The correct framing: stability is necessary, alignment is sufficient only given stability.

5.6 Encoder Learning Rate Sweep

A natural objection: perhaps drift is a tuning artifact — the free encoder simply uses the wrong learning rate. We test four LR values {1e-4, 3e-4, 1e-3, 3e-3} on Push-T with SIGReg, seed=42, 50 epochs.

Condition	Best Val Loss	Ratio vs prescribed	R ² (0→1)
Prescribed (LR=3e-4)	0.099	1.0×	1.0000
Free LR=1e-4	0.689	7.0×	0.8459
Free LR=3e-4	0.510	5.2×	0.7801

Condition	Best Val Loss	Ratio vs prescribed	$R^2(0 \rightarrow 1)$
Free LR=1e-3	0.467	4.7×	0.7600
Free LR=3e-3	0.420	4.3×	0.7437

Prescribed wins at every LR. The gap ranges from 4.3× to 7.0× — not a tuning problem. R^2 transfer on epoch 0→1 is below 0.85 for all free conditions, confirming that drift occurs regardless of learning rate. Higher LR produces better final loss (the encoder escapes bad regions faster) but does not eliminate the structural disadvantage.

5.7 EMA Target Encoder Baseline

EMA target encoders (as in I-JEPA, BYOL) provide slowly-moving targets to reduce representation drift. We test free encoder + EMA (decay=0.996) on the same Push-T setup.

Condition	Best Val Loss	Ratio vs prescribed
Prescribed	0.099	1.0×
Free + EMA	0.604	6.1×
Free (LR=3e-4)	0.510	5.2×

EMA does not help — it is 6.1× worse than prescribed and worse than plain free (5.2×). EMA stabilizes the *target* representation but does not fix the *online encoder's* coordinate system. The additional inertia slows adaptation without preventing drift. Prescribed axes are the limiting case of EMA with decay=1.0 and perfect initialization — but the benefit comes from fixing the right coordinates, not from slowing change.

5.8 Cross-Modal Validation: Rico UI (Vision)

All previous experiments use the Push-T state space. To test whether drift is domain-specific or structural, we measure it on Rico UI screenshots — a vision task with different architecture (Transformer) and different data (398 mobile UI screenshots).

Conditions match Push-T: Free encoder projects to 3D (matching prescribed dimensionality) + SIGReg. ShovJEPA provides prescribed 3D axes (position, functionality, depth from View Hierarchy). 100 epochs, seed=42.

Metric	Free 3D + SIGReg	ShovJEPA (prescribed 3D)
$R^2(0 \rightarrow 1)$	0.926	0.901
R^2 mean	0.988	0.996
Effective rank	1.62 → 2.38	1.24 → 1.17
Condition number	55 → 8	136 → 12,218

Drift in Rico is weaker than in Push-T (R^2 0.93 vs 0.78 on epoch 0→1) — consistent with the simpler task (binary classification, fewer samples). ShovJEPA $R^2(0 \rightarrow 1) = 0.90$

reflects the fact that the prescribed head (shown: Linear→GELU→Linear→Sigmoid) is itself learned, unlike Push-T where prescribed = direct coordinate extraction with no learnable parameters.

The key observation: SIGReg successfully prevents collapse in 3D (effective rank increases from 1.6 to 2.4), but the free encoder still does not converge to a stable coordinate system — eigenvalue trajectories fluctuate throughout training. Prescribed axes reach 100% validation accuracy; the free encoder does not.

Cross-modal validation confirms: drift is not a property of Push-T or state-space encoders. It occurs in vision with a Transformer architecture. The magnitude differs — weaker in Rico — but the structural pattern is the same.

6. Discussion

6.1 Reframing the Collapse Problem

The standard framing: “how do we prevent rank collapse in a free space?” In our setting, this is the wrong question. SIGReg successfully prevents rank collapse. The free encoder has full rank and high isotropy. And it still loses by one to two orders of magnitude. Rank collapse is not the dominant failure mode here; this does not imply that rank is irrelevant in all settings, but it demonstrates that full rank is not sufficient.

The more productive question, at least for this class of problems: “how do we maintain an identifiable, stable coordinate system that downstream modules can build on?” This is a different kind of problem — not geometric, but temporal and semantic.

A natural objection: why does the free encoder not converge to the same (or equivalent) coordinates as the prescribed encoder? The answer is that the JEPA prediction objective does not enforce temporal consistency of the representation. Multiple coordinate systems can yield equivalent prediction loss at any single training step. The optimizer is free to traverse this equivalence class, and in practice it does — rapidly in the early epochs, as our drift measurements show. The predictor, training simultaneously, cannot track this movement. Prescribed axes eliminate the equivalence class entirely: there is only one coordinate system, and every gradient step builds on it.

6.2 Why SIGReg Can Hurt

SIGReg forces the embedding distribution toward an isotropic Gaussian. Our eigenvalue analysis shows that the task structure is anisotropic: position coordinates (x , y) have more variance than angle (θ). SIGReg overrides this natural structure, spending encoder capacity on maintaining artificial isotropy rather than reflecting the task.

This explains the paradox from Lazarev (2026): SIGReg on a free encoder makes performance $4.2\times$ worse (free without SIGReg: 0.037, free with SIGReg: 0.156). SIGReg successfully prevents rank collapse — and simultaneously interferes with the emergence of a stable coordinate system in the encoder.

6.3 Connection to EMA and Stop-Gradient

The large-scale structural drift we observe is related to the instability that EMA target encoders and stop-gradient operations are designed to address in standard JEPA training. EMA provides a slowly-moving target that reduces the rate of coordinate change. Stop-gradient prevents the encoder from being updated through the prediction loss. Both are partial solutions to the drift problem — they slow the movement of the representation space without fixing it.

Prescribed axes are the limiting case: the representation space does not move at all. The gap between prescribed (0.0025) and the best freeze condition (0.065) quantifies how much is lost even with perfect stability, when the coordinates lack semantic alignment.

6.4 Connection to Adversarial State Gradients (GRASP)

The temporal non-identifiability we observe is related to a phenomenon studied in the planning literature. Psenka et al. (2026) prove (Theorem 1, GRASP) that any differentiable loss function over state/action trajectories in a learned world model cannot simultaneously (1) have minimizers corresponding to dynamically feasible trajectories and (2) be insensitive to the world model’s state gradient $\nabla_s F_\theta$. In other words, state gradients are unavoidable — and in high-dimensional learned spaces, they are adversarial: a small perturbation δ in state space ($\|\delta\| \ll 1$) can steer the world model to produce any desired output, regardless of the starting state.

GRASP addresses this at the planning level by applying stop-gradient to state inputs: gradients flow through actions but not through states. This prevents the planner from exploiting adversarial Jacobian structure.

Our drift analysis provides empirical evidence consistent with the same phenomenon at the representation learning level. The free encoder’s coordinate system changes so rapidly in early training that any module depending on it — predictor in our case, planner in GRASP’s case — receives incoherent gradient signal. Our R^2 transfer < -62 on epoch 0→1 is a direct measurement of this: a linear mapping from the representation to physical coordinates becomes adversarially wrong after one step of encoder optimization.

The two approaches address the same instability at different levels. GRASP masks the symptom: state gradients are detached during planning, so the planner cannot exploit the Jacobian. Prescribed axes remove the cause: the encoder has no learnable parameters, so the Jacobian $\nabla_s F_\theta$ does not exist — there is nothing to exploit. The approaches are complementary: prescribed axes stabilize the representation, GRASP stabilizes the planner. In principle, using both together would provide stability at both levels.

This connection also reframes our freeze test. Freezing the encoder at epoch 1 is a crude form of stop-gradient applied to training rather than planning: it prevents the encoder from generating adversarial state gradients for the predictor. The 20% improvement from freeze@1 is consistent with GRASP’s finding that removing state gradients improves optimization — and the remaining $>25\times$ gap to prescribed is con-

sistent with GRASP’s Theorem 1: stop-gradient alone does not guarantee feasible dynamics, it only prevents exploitation.

6.5 Limitations

All experiments use the Push-T environment with a 3D latent space. The drift dynamics may differ in higher-dimensional spaces or with different architectures. The freeze test uses a hard freeze (0% encoder gradient after epoch T); a soft schedule (gradual reduction) or reduced encoder learning rate might yield different results and would help quantify the relationship between drift rate and prediction quality more precisely.

The random fixed encoder control (Section 5.4) addresses the “privileged information” critique by showing that stability alone accounts for $17\times$ of the $233\times$ total gap. However, the random encoder projects from all 5 state dimensions — a different input subspace than prescribed, which uses only the 3 block coordinates. A more targeted control would test random projections from the same 3D subspace, or noisy versions of the prescribed coordinates. These experiments would further refine the boundary between the stability and alignment contributions.

We now test both an EMA target encoder baseline (Section 5.7) and an encoder learning rate sweep (Section 5.6). Neither eliminates the gap: EMA performs $6.1\times$ worse than prescribed (and worse than plain free), while prescribed wins at all four tested LR values ($4.3\text{--}7.0\times$). Cross-modal validation on Rico UI (Section 5.8) confirms drift in vision, though weaker than in Push-T.

The completed 2×2 factorial design (Section 5.5) shows that stability and alignment interact strongly — they are not independent factors. The stability effect is $330\times$ among aligned encoders but only $17.5\times$ among unaligned ones. This interaction means the original multiplicative decomposition ($17\times \times 13\times = 233\times$) was an approximation that obscured the hierarchical relationship between the factors. The correct structure — stability as prerequisite, alignment as additional benefit — is established in this setting but may differ in higher-dimensional spaces where drift dynamics could be qualitatively different.

7. Conclusion

In the setting studied here, the dominant failure mode of free representation spaces is not collapse of rank but loss of a temporally consistent coordinate parameterization. A free encoder can maintain full rank and high isotropy while its coordinate system drifts so rapidly that no downstream module can learn in it.

Prescribed axes solve both problems simultaneously: they provide coordinates that are stable (they never change) and aligned (they correspond to task-relevant quantities). The completed factorial design reveals the relationship between these factors: stability is a prerequisite, not merely a contributing factor. An encoder initialized on perfect coordinates but allowed to drift loses its entire advantage — performing no better than a randomly initialized free encoder. Conversely, a random but stable

projection outperforms the free encoder by $17\times$. Alignment contributes an additional $12\times$ — but only when stability is maintained. Rotated prescribed axes perform identically to prescribed ($1.09\times$), confirming that interpretability of axes is irrelevant — what matters is fixedness and subspace selection.

Whether this pattern generalizes beyond low-dimensional world models is an open question. But the result establishes a concrete, experimentally grounded alternative to the rank-collapse narrative: the goal of representation learning may be not to fill the space uniformly or use all dimensions, but to establish a coordinate system that is identifiable and stable enough for other modules to build on — and aligned enough with the task to make building worthwhile. An encoder learning rate sweep (4 values, $4.3\text{--}7.0\times$ gap), an EMA baseline ($6.1\times$ gap), and cross-modal validation on Rico UI vision confirm that this is a structural phenomenon, not a tuning artifact or domain-specific effect.

8. Reproducibility

Three seeds (42, 123, 777), 200 episodes per seed collected via gym-pusht with pymunk physics, 30 epochs, SIGReg $\lambda=0.09$. LR sweep: seed 42, 50 epochs, $\text{LR}=\{1\text{e-}4, 3\text{e-}4, 1\text{e-}3, 3\text{e-}3\} + \text{EMA}$ (decay=0.996). Rico UI drift: seed 42, 100 epochs, Rico dataset (500 samples), free 3D+SIGReg vs ShovJEPA. All analysis code, notebooks, data collection scripts, and results are available at <https://github.com/revenue7-eng/prescribed-axes-drift>.

References

- Assran, M., et al. (2023). Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. CVPR 2023.
- Balestriero, R., & LeCun, Y. (2025). LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics. arXiv:2511.08544.
- Bardes, A., Ponce, J., & LeCun, Y. (2022). VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. ICLR 2022.
- Lazarev, A. (2026). The Space Matters More Than the Loss: JEPA Collapse as a Problem of Structure, Not Optimization.
- LeCun, Y. (2022). A Path Towards Autonomous Machine Intelligence. Technical Report, Meta AI.
- Maes, L., Le Lidec, Q., Scieur, D., LeCun, Y., & Balestriero, R. (2026). LeWorld-Model: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. arXiv:2603.19312.
- Ozkara, B., et al. (2025). PAN: A World Model for General, Interactable, and Long-Horizon World Simulation. arXiv:2511.09057.

Psenka, M., Rabbat, M., Krishnapriyan, A., LeCun, Y., & Bar, A. (2026). Parallel Stochastic Gradient-Based Planning for World Models. arXiv:2602.00475.